HAI913I
Evolution et restructuration des logiciels

TP1 - Analyse statique

Nathan Boureux - M2 GL

N° étudiant : 22206444

Enseignant responsable : Bachar Rima

Année universitaire 2026–2027



Table des matières

1 Objectif et périmètre **1**

2 Conception générale **1**

3 Réponses **2**

3.1 Partie A 2

3.1.1 A1 2

3.1.2 A2 2

3.1.3 A3 2

3.1.4 B1 2

3.1.5 B2 2

3.1.6 B3 3

3.1.7 B4 3

4 Architecture et diagrammes **3**

5 Validation **4**

5.1 Étape A0 4

5.2 Étape A1 4

5.3 Étape A2 5

5.4 Étape A3 6

5.5 Étape B2 7

5.6 Étape B3 8

5.7 Étape B4 8

6 Difficultés **8**

7 Utilisation de l'IA **9**

8 Regard critique **9**

9 Conclusion **9**

1 Objectif et périmètre

L'outil développé au cours de ce tp est une outil d'analyse statique de code Java, utilisable en ligne de commande. L'objectif est d'extraire la structure du code (classes, méthodes, attributs, etc), de modéliser les appels de méthodes, de calculer un ensemble de métriques logicielles, et de générer un graphe d'appels.

L'outil prend en charge les objets Java standards (classes, interfaces, énumérations, etc), les invocations de méthodes classiques et de super-classes et les appels de constructeurs.

2 Conception générale

L'architecture repose sur trois parties distinctes :

— Analyse syntaxique (JDT) : Utilisation de Eclipse JDT pour parser les fichiers sources en Arbres de Syntaxe Abstraite (AST) avec résolution des liaisons (bindings).
— Extraction et Modélisation (Visiteurs) : Deux passes successives avec deux visiteurs différents (StructureVisitor puis CallVisitor). Ils parcourent l'AST pour instancier un modèle de faits personnalisé composé de POJO (ProjectFacts, TypeFact, MethodFact, CallFact).
— Exploitation (Services) : Les métriques et la génération du graphe utilisent exclusivement le modèle de faits, pour ne pas réutiliser l'API JDT (ne pas avoir ni à regénérer les AST, ni à les parcourir).

3 Réponses

3.1 Partie A

3.1.1 A1

La création de l'arbre a été faite via un simple ASTVisitor surchargeant preVisit pour indenter l'affichage en fonction de la profondeur du nœud.

Rôle de la valeur renvoyée par visit(...), et ce que change return false :

Dans JDT, l'infrastructure visite les enfants d'un nœud si et seulement si la méthode visit a renvoyé true.

Renvoyer false "supprime" le sous-arbre : le visiteur ne descendra pas dans les enfants de ce nœud.

3.1.2 A2

La classe StructureVisitor intercepte les TypeDeclaration, EnumDeclaration, etc. Pour différencier classes et interfaces, la méthode node.isInterface() est évaluée. Les constructeurs sont identifiés nativement via MethodDeclaration.isConstructor()

3.1.3 A3

La classe CallVisitor intercepte les MethodInvocation et SuperMethodInvocation. La difficulté majeure a été de déterminer si un appel était interne ou externe. La solution a consisté à résoudre le binding de la méthode (resolveMethodBinding()), puis à extraire sa classe déclarante, et enfin à évaluer IMethodBinding.getDeclaringClass().isFromSource()

3.1.4 B1

Création des objets du modèle de faits. La construction se fait en deux passes de visite, d'abord StructureVisitor puis Callvisitor, qui instancient puis remplissent le modèle de faits.

3.1.5 B2

Pour calculer le nombre de lignes par méthode sans inclure l'accolade fermante (qui fausse les moyennes), j'ai récupéré la position du nœud Block, calculé la position du dernier caractère utile (startPosition + length - 1), puis converti ces index en numéros de ligne via CompilationUnit.getLineNumber().

Pour extraire les 10% supérieurs en gérant les ex æquo, j'ai implémenté une méthode générique. Elle trie les éléments par ordre décroissant, calcule un index avec la valeur entière de la fonction plafond (n + 9) / 10, récupère la valeur à cet index, puis filtre toute la liste avec la condition valeur >= seuil et valeur > 0.

3.1.6 B3

J'ai utilisé deux structures Map<String, Integer Map<String,» : outgoingEdges (arcs sortants) et incomingEdges (arcs entrants). Lors du parcours des CallFact, si un appel interne est détecté, le poids de l'arc correspondant est incrémenté via Map.merge(cible, 1, Integer : :sum).

J'ai opté pour le format DOT pour l'export du graphe d'appel pour la facilité de production d'un graphe directement exploitable pour le rapport (fichier png). Le format DOT gère nativement les nœuds, les arcs orientés et leurs poids, et peut être converti très facilement en image png, sans nécessiter le développement d'un parseur ou d'une interface graphique supplémentaire.

3.1.7 B4

Le point d'entrée Main valide le nombre d'arguments et l'existence du dossier via l'API java.nio.file.Files. Les erreurs de compilation du code source analysé sont interceptées en itérant sur CompilationUnit.getProblems(). Si le problème renvoie isError() == true, il est loggué dans System.err, mais le visiteur est tout de même exécuté, garantissant une analyse en best-effort. Les erreurs de parsing d'arguments (comme un X non numérique) sont capturées via des try-catch et terminent le programme avec System.exit(2).

4 Architecture et diagrammes

L'application est structurée en cinq packages principaux :

— tp1 : Contient le Main (CLI et orchestration)
— tp1.parse : Configuration de l'analyseur JDT
— tp1.visitor : Contient StructureVisitor, CallVisitor et PrintTreeVisitor
— tp1.model : Les entités du modèle de faits
— tp1.metrics : Service de calculs et d'affichage des métriques
— tp1.callgraph : Service de construction du graphe sous la forme d'un fichier DOT

5 Validation

5.1 Étape A0

```
nathan@Lapointe:~/S9/ERL/TP_analyse_statique/starter$ java -jar target/hai913i-tp1-analyzer.jar ../validation/ tree
Racine des sources     : /home/nathan/Fac/2026/Cours/S9/ERL/TP_analyse_statique/validation/src
Unites de compilation  : 14
Erreurs de compilation : 0
```

On a bien 14 unités de compilation et 0 erreur sur le projet de validation

5.2 Étape A1

```
--- Arbre AST pour : Dvd.java ---
CompilationUnit
  PackageDeclaration
    QualifiedName
      SimpleName
      SimpleName
  TypeDeclaration
    Javadoc
    Modifier
    SimpleName
    SimpleType
      SimpleName
    FieldDeclaration
      Modifier
      Modifier
      PrimitiveType
      VariableDeclarationFragment
        SimpleName
    MethodDeclaration
      Modifier
      SimpleName
      SingleVariableDeclaration
        SimpleType
          SimpleName
        SimpleName
      SingleVariableDeclaration
        SimpleType
          SimpleName
        SimpleName
      SingleVariableDeclaration
        PrimitiveType
        SimpleName
      Block
        SuperConstructorInvocation
          SimpleName
          SimpleName
          QualifiedName
            SimpleName
            SimpleName
        ExpressionStatement
          Assignment
            FieldAccess
              ThisExpression
              SimpleName
            SimpleName
    MethodDeclaration
      Modifier
      PrimitiveType
      SimpleName
      Block
        ReturnStatement
          SimpleName
    MethodDeclaration
      MarkerAnnotation
        SimpleName
      Modifier
      PrimitiveType
      SimpleName
      Block
        ReturnStatement
          NumberLiteral
```

```
--- Arbre AST pour : Category.java -
CompilationUnit
  PackageDeclaration
    QualifiedName
      SimpleName
      SimpleName
  EnumDeclaration
    Javadoc
    Modifier
    SimpleName
    EnumConstantDeclaration
      SimpleName
      StringLiteral
    EnumConstantDeclaration
      SimpleName
      StringLiteral
    EnumConstantDeclaration
      SimpleName
      StringLiteral
    FieldDeclaration
      Modifier
      Modifier
      SimpleType
        SimpleName
      VariableDeclarationFragment
        SimpleName
    MethodDeclaration
      SimpleName
      SingleVariableDeclaration
        SimpleType
          SimpleName
        SimpleName
      Block
        ExpressionStatement
          Assignment
            FieldAccess
              ThisExpression
              SimpleName
            SimpleName
    MethodDeclaration
      Modifier
      SimpleType
        SimpleName
      SimpleName
      Block
        ReturnStatement
          SimpleName
```

Pour Dvd.java, l'arbre montre bien un TypeDeclaration, un FieldDeclaration et trois MethodDeclaration (dont le constructeur, qui contient un SuperConstructorInvocation). Pour Category.java, la racine du type est un EnumDeclaration

5.3 Étape A2

```
nathan@Lapointe:~/S9/ERL/TP_analyse_statique/starter$ java -jar target/hai913i-tp1-analyzer.jar ../validation/ structure
Racine des sources     : /home/nathan/Fac/2026/Cours/S9/ERL/TP_analyse_statique/validation/src
Unites de compilation  : 14
Erreurs de compilation : 0

--- BILAN DE STRUCTURE ---
15 classes au total : 12 classes, 2 interfaces, 1 énumérations
59 méthodes, dont 9 constructeurs
21 attributs
```

On a bien 15 classes : 12 classes, 2 interfaces, 1 énumération ; 59 méthodes, dont 9 constructeurs ; 21 attributs dans le projet de validation

```
--- CLASSE : library.service.LoanException ---
  Hérite de : java.lang.Exception
  Hérite de : java.lang.Throwable
  Méthode : LoanException | Params : 1 | Constructeur

--- CLASSE : library.service.LoanService ---
  Attribut : MAX_LOANS | Type : int | Visibilité : public
  Attribut : catalog | Type : Catalog | Visibilité : private
  Attribut : notifier | Type : Notifier | Visibilité : private
  Attribut : history | Type : List<Loan> | Visibilité : private
  Attribut : refusals | Type : int | Visibilité : package
  Méthode : LoanService | Params : 2 | Constructeur
  Méthode : borrow | Params : 3
  Méthode : giveBack | Params : 1
  Méthode : totalFees | Params : 0
  Méthode : getHistory | Params : 0
  Méthode : getRefusals | Params : 0

--- CLASSE : library.service.LoanService.Loan ---
  Attribut : item | Type : Item | Visibilité : private
  Attribut : member | Type : Member | Visibilité : private
  Attribut : days | Type : int | Visibilité : private
  Méthode : Loan | Params : 3 | Constructeur
  Méthode : getItem | Params : 0
  Méthode : getMember | Params : 0
  Méthode : getDays | Params : 0
```

LoanService : 6 méthodes et 5 attributs, LoanService.Loan : 4 méthodes et 3 attributs, Superclasses de library.service.LoanException : java.lang.Exception, java.lang.Throwable

```
--- CLASSE : library.model.Item ---
  Attribut : MAX_LOAN_DAYS | Type : int | Visibilité : public
  Attribut : id | Type : String | Visibilité : protected
  Attribut : title | Type : String | Visibilité : protected
  Attribut : borrower | Type : Member | Visibilité : private
  Attribut : category | Type : Category | Visibilité : package
  Méthode : Item | Params : 3 | Constructeur
  Méthode : getId | Params : 0
  Méthode : getTitle | Params : 0
  Méthode : getCategory | Params : 0
  Méthode : isAvailable | Params : 0
  Méthode : checkOut | Params : 1
  Méthode : giveBack | Params : 0
  Méthode : describe | Params : 0
  Méthode : dailyRate | Params : 0
  Méthode : fee | Params : 1
```

Attributs de library.model.Item : MAX_LOAN_DAYS (public), id et title (protected), borrower (private), category (package)

5.4 Étape A3

```
nathan@Lapointe:~/S9/ERL/TP_analyse_statique/starter$ java -jar target/hai913i-tp1-analyzer.jar ../validation/ calls
Racine des sources     : /home/nathan/Fac/2026/Cours/S9/ERL/TP_analyse_statique/validation/src
Unites de compilation  : 14
Erreurs de compilation : 0

--- BILAN DES APPELS ---
94 appels (93 MethodInvocation et 1 SuperMethodInvocation)
46 vers des méthodes du projet (internes)
48 vers l'extérieur (externes)
0 non résolus
```

On a bien 94 appels (93 MethodInvocation et 1 SuperMethodInvocation) : 46 vers des méthodes du projet, 48 vers l'extérieur, 0 non résolus dans le projet de validation

```
[LoanService.java | Ligne 36] LoanService#borrow ---> appelle ---> library.model.Loanable#checkOut
    (Receveur : library.model.Loanable | Nature : Interne)
```

LoanService.borrow, ligne 36, loanable.checkOut(member) : receveur library.model.Loanable

```
[LibraryApp.java | Ligne 32] LibraryApp#main ---> appelle ---> library.model.Item#getTitle
    (Receveur : library.model.Book | Nature : Interne)
```

LibraryApp.main, ligne 32, getTitle() : receveur library.model.Book, cible library.model.Item#getTitle()

```
[Book.java | Ligne 32] Book#describe ---> appelle ---> library.model.Item#describe
    (Receveur : library.model.Item | Nature : Interne)
```

Book.describe, ligne 32, super.describe() : receveur library.model.Item

```
[Catalog.java | Ligne 37] Catalog#available ---> appelle ---> library.model.Item#isAvailable
    (Receveur : library.model.Item | Nature : Interne)
```

Catalog.available, ligne 37, item.isAvailable() : receveur library.model.Item

```
nathan@Lapointe:~/S9/test/TP_analyse_statique/starter$ java -jar target/hai913i-tp1-analyzer.jar ../robustness/missing-dep/ calls
Greeter.java:3 The import org.apache cannot be resolved
Greeter.java:14 StringUtils cannot be resolved
Racine des sources     : /home/nathan/Fac/2026/Cours/S9/test/TP_analyse_statique/robustness/missing-dep/src
Unites de compilation  : 2
Erreurs de compilation : 2
Ligne 14 : capitalize -> NON RÉSOLU
[Greeter.java | Ligne 15] Greeter#greet ---> appelle ---> demo.Formatter#decorate
    (Receveur : demo.Formatter | Nature : Interne)


--- BILAN DES APPELS ---
2 appels (2 MethodInvocation et 0 SuperMethodInvocation)
1 vers des méthodes du projet (internes)
0 vers l'extérieur (externes)
1 non résolus
```

Sur resources/robustness/missing-dep : l'appel à StringUtils.capitalize est signalé non résolu, l'appel à formatter.decorate est résolu, et l'analyse va jusqu'au bout

5.5 Étape B2

```
nathan@Lapointe:~/S9/ERL/TP_analyse_statique/starter$ java -jar target/hai913i-tp1-analyzer.jar ../validation/ metrics
Racine des sources     : /home/nathan/Fac/2026/Cours/S9/ERL/TP_analyse_statique/validation/src
Unites de compilation  : 14
Erreurs de compilation : 0

--- MÉTRIQUES ---
1. Nombre de classes : 15
2. Lignes de code de l'application : 493
3. Nombre total de méthodes : 59
4. Nombre total de paquetages : 4
5. Moyenne de méthodes par classe : 3.93
6. Moyenne de lignes par méthode : 4.64
7. Moyenne d'attributs par classe : 1.40
8. Les 10% des classes avec le plus de méthodes : library.model.Item, library.model.Member, library.service.LoanService
9. Les 10% des classes avec le plus d'attributs : library.model.Item, library.service.LoanService
10. Classes dans les deux catégories précédentes : library.model.Item, library.service.LoanService
11. Classes avec plus de X=5 méthodes : library.model.Item, library.model.Member, library.service.LoanService
12. Les 10% des méthodes avec le plus de lignes par classe :
    - library.app.LibraryApp : main
    - library.model.Book : Book
    - library.model.Category : Category, getLabel
    - library.model.Dvd : Dvd
    - library.model.Item : checkOut
    - library.model.Member : Member, getName, addLoan, removeLoan, loanCount, getLoans
    - library.service.Catalog : available
    - library.service.ConsoleNotifier : send
    - library.service.LoanException : LoanException
    - library.service.LoanService : borrow
    - library.service.LoanService.Loan : Loan
    - library.util.Statistics : mean, report
    - library.util.TextUtils : capitalize, repeat, pad
13. Nombre maximal de paramètres (5) : library.util.Statistics#report(java.lang.String,int,int,double,int)
```

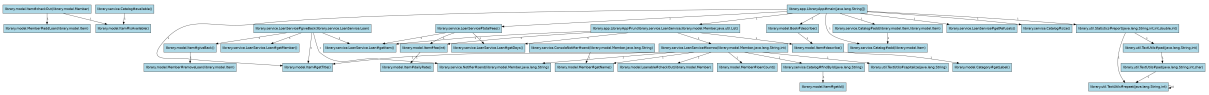
On a bien les mêmes métriques que sur le sujet

5.6 Étape B3

```
nathan@Lapointe:~/S9/ERL/TP_analyse_statique/starter$ java -jar target/hai913i-tp1-analyzer.jar ../validation/ graph
Racine des sources     : /home/nathan/Fac/2026/Cours/S9/ERL/TP_analyse_statique/validation/src
Unites de compilation  : 14
Erreurs de compilation : 0

--- GRAPHE D'APPEL ---
59 nœuds, 44 arcs, 46 sites d'appel internes
48 appels externes, 0 appels non résolus
Graphe exporté avec succès : callgraph.dot
```

59 nœuds, 44 arcs, 46 sites d'appel internes, 48 appels externes, 0 appel non résolu



Le graphe d'appel généré en dot puis transformé en png pour être affiché. Catalog#add(Item,Item) → Catalog#add(Item) de poids 2 ; TextUtils#repeat s'appelle lui-même. Appelantes de Loanable#checkOut(Member) : LoanService#borrow seulement ; Item#checkOut(Member) n'a aucune appelante

5.7 Étape B4

```
nathan@Lapointe:~/S9/ERL/TP_analyse_statique/starter$ java -jar target/hai913i-tp1-analyzer.jar ../robustness/no-java-files/ structure
Erreur : Aucun fichier source .java n'a été trouvé dans le dossier : ../robustness/no-java-files
nathan@Lapointe:~/S9/ERL/TP_analyse_statique/starter$ java -jar target/hai913i-tp1-analyzer.jar ../robustness/syntax-error/ structure
Broken.java:10 Syntax error, insert ";" to complete BlockStatements
Broken.java:11 Syntax error, insert "}" to complete ClassBody
Racine des sources     : /home/nathan/Fac/2026/Cours/S9/ERL/TP_analyse_statique/robustness/syntax-error/src
Unites de compilation  : 2
Erreurs de compilation : 2

--- BILAN DE STRUCTURE ---
2 classes au total : 2 classes, 0 interfaces, 0 énumérations
2 méthodes, dont 0 constructeurs
0 attributs
nathan@Lapointe:~/S9/ERL/TP_analyse_statique/starter$ java -jar target/hai913i-tp1-analyzer.jar ../robustness/syntax-error/ metrics abc
Broken.java:10 Syntax error, insert ";" to complete BlockStatements
Broken.java:11 Syntax error, insert "}" to complete ClassBody
Racine des sources     : /home/nathan/Fac/2026/Cours/S9/ERL/TP_analyse_statique/robustness/syntax-error/src
Unites de compilation  : 2
Erreurs de compilation : 2
Erreur : Le seuil X doit être un entier valide.

Usage : java -jar target/hai913i-tp1-analyzer.jar <DOSSIER_DU_PROJET> <ACTION>
Actions disponibles :
  tree      : Arbre AST (Étape A1)
  structure : Structure (Étape A2)
  calls     : Appels de méthodes (Étape A3)
  metrics   : Calcul des 13 métriques (Étape B2) [SEUIL_X optionnel, par défaut=5]
  graph     : Génération du graphe d'appel et export DOT (Étape B3)
```

Les tests de robustesse passent tous

6 Difficultés

La détermination de la portée des appels a posé des difficultés. Les initialisations d'attributs au niveau de la classe (ex : List<String> list = new ArrayList<>() ;) génèrent des nœuds d'invocation en dehors de tout bloc de méthode. Pour pallier ce problème, un mécanisme d'état (currentMethodId) a été implémenté dans le CallVisitor pour ignorer ces appels, respectant ainsi la sémantique stricte d'un graphe d'appel de méthodes.

7 Utilisation de l'IA

Un modèle LLM (Gemini 3.1 Pro) a été utilisé ponctuellement comme assistant de développement :

Pour la génération rapide des classes pour le modèle de faits, l'aide au développement de l'algorithme textuel des 10%, et aide pour l'export en DOT.

Tout le code généré a bien sûr été vérifié et modifié/corrigé manuellement, car l'IA proposait parfois des suggestions qui n'avaient aucun rapport avec le sujet. L'architecture en "deux passes" (Structure puis Appels) est un choix proposé par l'IA, le reste de la conception est personnelle.

8 Regard critique

L'analyse statique pure développée ici fonctionne, mais on en détecte aussi les limites :

— L'outil identifie le type statique du receveur. Dans le cas d'une interface ayant de multiples implémentations, le graphe d'appel ne trace l'arc que vers la méthode de l'interface, ignorant le comportement dynamique réel, ce qui est normal mais donc ne suffit pas pour une analyse complète.
— Les métriques ne sont pas forcément toutes pertinentes : Les lignes de code physiques (Files.readAllLines) sont fortement influencées par le style de formatage. Une métrique basée sur les lignes logiques (nombre d'instructions ou de points-virgules) ou sur le nombre de blocs complexes (if, while, switch, etc) fournirait une évaluation beaucoup plus fiable de la maintenabilité des méthodes

9 Conclusion

Ce projet a permis de construire un outil d'analyse statique fonctionnel, robuste et respectueux des principes d'architecture logicielle (séparation des préoccupations). Le remplacement progressif d'une analyse directe sur l'AST par un modèle de faits intermédiaire a prouvé son efficacité, permettant de diminuer la quantité de calculs. L'outil valide l'intégralité des points de contrôle et est une bonne base pour les prochains TP.

10 Source

Dépot : https://github.com/NathanBrx/TP_analyse_statique